LLM Pruning and Distillation in Practice: The Minitron Approach

Sharath Turuvekere Sreenivas*, Saurav Muralidharan*, Raviraj Joshi, Marcin Chochowski, Ameya Sunil Mahabaleshwarkar, Gerald Shen, Jiaqi Zeng, Zijia Chen, Yoshi Suhara, Shizhe Diao, Chenhan Yu, Wei-Chun Chen, Hayley Ross, Oluwatobi Olabiyi, Ashwath Aithal, Oleksii Kuchaiev, Daniel Korzekwa, Pavlo Molchanov, Mostofa Patwary, Mohammad Shoeybi, Jan Kautz, and Bryan Catanzaro

Abstract: Structured pruning with knowledge distillation is a potent combination for obtaining small language models (SLMs) with significantly fewer training tokens and compute resources compared to training from scratch. In this work, we investigate how this strategy can be effectively applied in instances where access to the the original pretraining dataset is restricted. We introduce a new *teacher correction* phase before distillation which lets the teacher model adjust to our specific data distribution using a lightweight fine-tuning phase. We apply this strategy to compress the Mistral NeMo 12B and Llama 3.1 8B models to 8B and 4B parameters, respectively, using pruning and distillation. We explore two distinct pruning strategies: (1) depth pruning and (2) joint hidden/attention/MLP (width) pruning, and evaluate the results on common benchmarks from the LM Evaluation Harness. The models are then aligned with NeMo Aligner and further tested for instruction following, role-play, math, coding and function calling capabilities. This approach produces the state-of-the-art Mistral-NeMo-Minitron-8B (MN-Minitron-8B for brevity) model from Mistral NeMo 12B, and a compelling 4B model from Llama 3.1 8B. We open-source our base model weights on Hugging Face with a permissive license.

Models on Hugging Face: Mistral-NeMo-Minitron-8B-Base | Llama-3.1-Minitron-4B-Width-Base | Llama-3.1-Minitron-4B-Depth-Base

arXiv:2408.11796v4 [cs.CL] 9 Dec 2024

Introduction

LLM providers often train an entire family of models from scratch, each with a different size (number of parameters, e.g. Llama 3.1 with 8B, 70B, and 405B parameters [1]); this is done to aid users targeting different deployment scales, sizes and compute budgets. However, training multiple billion-plus parameter models from scratch is extremely time-, data- and resource-intensive. Recent work has demonstrated the effectiveness of combining weight pruning with knowledge distillation to significantly reduce the cost of training LLM model families [2]. Here, only the biggest model in the family is trained from scratch; other models are obtained by successively pruning the bigger model(s) and then performing knowledge distillation [3] to recover the accuracy of pruned models. While highly effective, this line of work assumes access to the original pretraining dataset for the distillation phase. With a growing number of frontier LLMs (including open ones) being trained on private, proprietary datasets [1, 4], this assumption often fails to hold.

In this work, we adapt the original Minitron compression recipe [2] along two directions: (1) we introduce a new *teacher correction* phase for adapting the teacher (unpruned) model to our own data distribution, thus removing any need to access the original pretraining dataset, and (2) we introduce a new and more effective downstream task-based saliency criteria for depth pruning. We successfully apply our updated compression strategy to two state-of-the-art models: Llama 3.1 8B [1] and Mistral NeMo 12B [5], compressing them down to 4B and 8B parameters, respectively. For Llama 3.1 8B, we produce two distinct compressed models in the 4B parameter range: (1) Llama 3.1-Minitron-4B-Width (pruning only the width axes), and (2) Llama 3.1-Minitron-

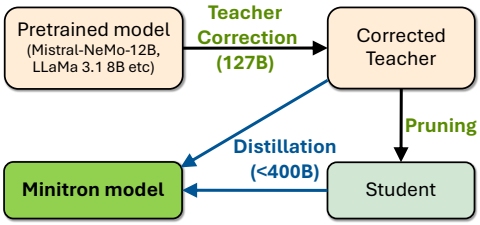

Figure 1 | High-level overview of our proposed pruning and distillation approach. The total number of tokens used for each step is indicated in parentheses.

* Equal contribution.

© 2024 NVIDIA. All rights reserved.

Benchmarks (shots)	Gemma2 2B*	Minitron 4B	**Llama-3.1-Minitron 4B-Depth**	**Llama-3.1-Minitron 4B-Width**	Gemma 7B	Mistral 7B	Llama 3.1 8B	**MN-Minitron 8B**	Mistral NeMo 12B-Base	Mistral NeMo 12B-FT
Total Params	2.6B	4.2B	4.5B	4.5B	8.5B	7.3B	8B	8.4B	12.2B	12.2B
Non-Emb. Params	2B	2.6B	3.7B	3.7B	7.7B	7B	7B	7.3B	10.9B	10.9B
Training Tokens	2T	**94B**	**94B**	**94B**	6T	8T	15T	**380B**	-	+0.1T
Winogrande(5)	70.9	**74.0**	72.1	73.5	78	78.5	77.3	**80.4**	82.2	82.7
Arc_challenge(25)	55.4	50.9	52.6	**55.6**	61	60.3	57.9	**64.4**	65.1	62.3
MMLU(5)	51.3	58.6	58.7	**60.5**	64	64.1	65.3	**69.5**	69.0	70.1
Hellaswag(10)	73.0	75.0	73.2	**76.1**	82	**83.2**	81.8	83.0	85.2	85.3
GSM8k(5)	23.9	24.1	16.8	**41.2**	50	37.0	48.6	**58.5**	56.4	55.7
Truthfulqa(0)	-	**42.9**	38.2	**42.9**	45	42.6	45.0	**47.6**	49.8	48.3
XLSum en(20%) (3)	-	**29.5**	27.2	28.7	17	4.8	30.0	**32.0**	33.4	31.9
MBPP(0)	29.0	28.2	30.7	**32.4**	39	38.8	42.3	**43.8**	42.6	47.9
HumanEval(n=20)(0)	20.1	**23.3**	-	-	32.0	28.7	24.8	**36.2**	23.8	23.8

Table 1 | Accuracy numbers for our MN-Minitron-8B and Llama 3.1-Minitron-4B models. We compare our models to similarly-sized SoTA open models on a variety of common language modeling benchmarks. All evaluations are conducted by us, except entries marked with * (taken from corresponding papers).

Benchmarks (shots)	Phi-2 2.7B	Gemma2 2B	Qwen2 1.5B	Minitron 4B	**Llama-3.1-Minitron 4B-Depth**	**Llama-3.1-Minitron 4B-Width**	LLama 3.1 8B	**MN-Minitron 8B**
MT-Bench (GPT4-Turbo)	5.14	**7.44**	5.49	6.46	6.19	6.88	7.78	**7.86**
MMLU (5)	56.8	56.9	55.6	59.3	**61.21**	59.89	69.4*	**70.4**
GSM8K (0)	19.9	52.2	27.2	65.1	71.11	**79.76**	83.8	**87.1**
GPQA (0)	28.8	25.9	28.1	29.5	**32.59**	30.36	30.4*	**31.5**
HumanEval (0)	**47.6***	45.1	47.0*	39.6	42.7	47.0	**72.6**	71.3
MBPP (0)	55.0*	50.4	51.9*	57.4	60.3	**65.1**	**72.8***	72.5
IFEval	44.0	64.5	39.8	75.3	66.77	**79.54**	80.4*	**84.4**
BFCLv2 (Live)	38.7	40.2	39.9	53.1	**55.89**	55.0	44.3	**67.6**

Table 2 | Accuracy numbers for instruction tuned models on a variety of benchmarks. All evaluations are conducted by us, except entries marked with * (taken from corresponding papers). Best of each section in **bold**. For IFEval, we report the average of prompt and instruction across loose and strict evaluations. For BFCLv2, we report live accuracy only.

4B-Depth (pruning depth only). Figure 1 provides a high-level overview of our approach.

Tables 1 and 2 provide a summary of our results: our compression strategy yield a state-of-the-art 8B model (MN-Minitron-8B) which outperforms all similarly-sized models across the board on common language modeling benchmarks. Our Llama 3.1-Minitron-4B models (both depth and width-pruned variants) also exhibit strong accuracy compared to the teacher Llama 3.1 8B model and the previous-generation Minitron-4B model [2]; among the two variants, the width-pruned variant achieves better overall accuracy than the depth-pruned one. In terms of runtime inference performance measured using TensorRT-LLM, the Llama 3.1-Minitron-4B models provide an average speedup of 2.7× and 1.8× for the depth and width pruned variants, respectively, compared to the original Llama 3.1 8B model.

Methodology

A high-level overview of our approach is illustrated in Figure 1. Here, the teacher model undergoes a lightweight adjustment phase on the target dataset to be used for distillation - we refer to this step as *teacher correction*. Next, pruning is applied to compress the model, following which distillation is used to recover model accuracy.

Teacher Correction

Distillation is an effective technique to condense knowledge from a more accurate teacher model to improve a less accurate student model [3] [2]. Typically, knowledge is distilled using the same dataset the teacher model was trained on. In cases where access to the original training data is restricted, we notice from our experiments that the teacher model provides sub-optimal guidance if a different dataset is used to distill the knowledge. We hypothesize this is due to the change in distribution of sub-word tokens across the original dataset the teacher model was trained on vs. the dataset being distilled on. To this end, we propose a novel teacher correction phase (illustrated in Figure 2), where we perform a lightweight (∼100B tokens) fine-tuning of the teacher model to adapt to the new distillation dataset. We demonstrate in our experimental evaluation (Figure 4 in particular) that

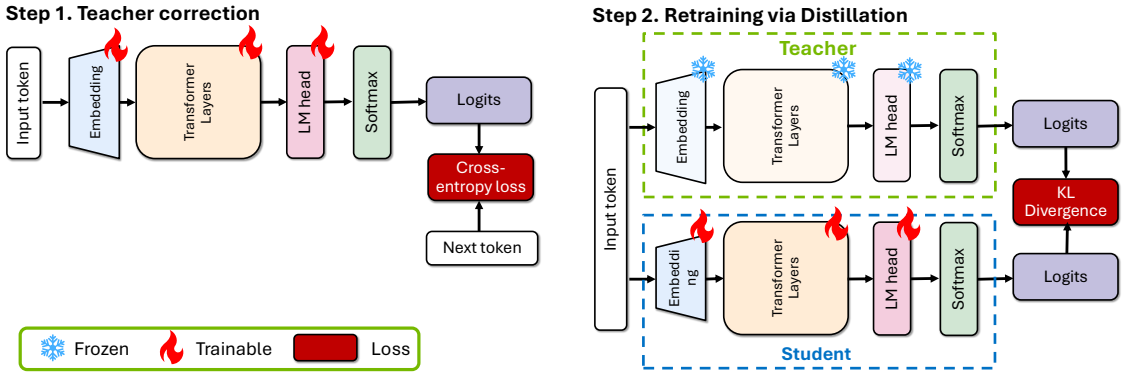

Figure 2 | Overview of distillation: if/when the original training data is unavailable, a lightweight fine-tuning of the original model on the distillation dataset is recommended, to be used as a teacher. Distillation is then performed by minimizing KL divergence on the logits of the teacher and the pruned student model.

this procedure significantly improves the guidance resulting in a more accurate student model. We also explore correcting the teacher in parallel to distillation, and demonstrate that this performs on par with using guidance from a fully corrected teacher.

Pruning

Weight pruning is a powerful and well-known technique for reducing model size. In this work, we focus on structured pruning, where blocks (or channels) of nonzero elements are removed at once from model weights; examples of structured pruning techniques include neuron, attention head, convolutional filter, and depth pruning [6, 7, 8, 9]. We follow the pruning recipe outlined in Minitron [2]: as shown in Figure 3, we start the pruning process by first computing the importance of each layer, neuron, head, and embedding dimension. We then sort these importance scores to compute a corresponding importance ranking.

Importance Estimation We use a purely activation-based importance estimation strategy that simultaneously computes sensitivity information for all the axes we consider (depth, neuron, head, and embedding channel) using a small calibration dataset and only forward propagation passes. We consider depth pruning as a special case and do not combine it with compressing other dimensions. We compute the importance of each head, neuron and embedding channel by examining the activations produced by the multi-head attention (MHA), multi-layer perceptron (MLP) and LayerNorm layers, respectively. We use a small calibration dataset (1024 samples) drawn randomly from the full dataset for this purpose.

Layer Importance For depth pruning, we consider two distinct metrics for evaluating layer importance: (1) LM validation loss/PPL, and (2) accuracy on the downstream task. We do not consider the Block Importance (BI) metric [8] as it was recently shown to under-perform the validation loss/PPL metric [2]. For ranking, we simply remove a single or a block of contiguous layers and compute its effect on each metric; this serves as the "importance" or sensitivity of the layer/layer block. Based on our empirical analysis (see Figures 8 and 9), we use the Winogrande metric [10] to prune sets of contiguous layers. This pruning strategy evolved from two important observations: (1) LM validation loss/PPL-based layer importance fails to produce the most accurate pruned model(s) on downstream tasks, and (2) dropping contiguous layers is better than individual, as also observed in Gromov et al. [11].

Model Trimming Following Minitron [2], for a given architecture configuration, we first rank the elements of each axis according to the computed importance and perform trimming of the corresponding weight matrices directly. For neuron and head pruning, we trim MLP and MHA layer weights, respectively. In the case of embedding channels, we trim the embedding dimension of the weight matrices in MLP, MHA, and LayerNorm layers. The original approach uses Neural Architecture Search (NAS) to find the best architecture; in this work, we skip this step and instead utilize the network architecture-related learnings from the original paper.

Retraining with Distillation

We use the term retraining to refer to the accuracy recovery process post pruning. In this work, we explore two retraining strategies: (1) conventional training, leveraging ground truth labels, and (2) knowledge distillation using supervision from the unpruned model (teacher). Knowledge Distillation (KD) [3] involves transfer of knowledge from a larger or more complex model called the teacher to a smaller/simpler

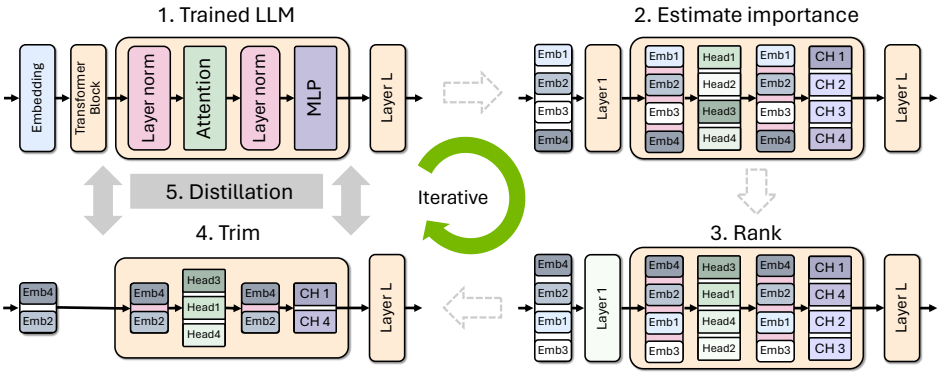

Figure 3 | Pruning and distillation process outlined in the original paper [2]. We follow the same approach in this work.

model called the student. The knowledge transfer is achieved by having the student model mimic the output and/or the intermediate states of the teacher model. In our case, the uncompressed and pruned models correspond to the teacher and student, respectively. Following the best practices outlined in the Minitron work [2], we use forward KL Divergence loss [12] on the teacher and student logits only; this is illustrated in Figure 2.

	LLaMa-3.1-Minitron 4B-Width	LLaMa-3.1-Minitron 4B-Depth	MN-Minitron 8B
Total params	4.5B	4.5B	8.4B
Non-Emb params	3.7B	3.5B	7.3B
Hidden size	3072	4096	4096
Vocabulary	128256	128256	131072
MLP hidden dim	9216	14336	11520
Depth	32	16	40
Attention groups	8	8	8
Query heads	32	32	32
Head dimension	128	128	128

Table 3 | Architecture details of our compressed models.

Training Details

Pre-training

Llama 3.1 8B [1] and Mistral NeMo 12B [5] are pretrained on different proprietary datasets, which we do not have access to. According to the Llama 3.1 tech report [1], the 8B model is pretrained on 15T tokens. We start with the corresponding Base models that are openly available on Hugging Face.

Dataset We use the Nemotron-4 curated continued training (CT) dataset [13] [14] for all our pruning and distillation experiments.

Teacher Correction

Using the original Mistral NeMo 12B or Llama 3.1 8B models directly as a teacher performs sub-optimally on our dataset. To counter this, we apply teacher correction, as described in the previous section, to both models with $\sim 100B$ tokens. Since the goal is to adapt the teacher model to the distillation dataset, we use 120 steps of warm-up and low learning rates: one-fifth the peak learning rate, identical batch size, minimum learning rate and decay schedule the original model was trained on. We notice that the correction process has a minor effect on the teacher model's accuracy on downstream tasks, with some tasks improving and some degrading as shown in Table 1. We hypothesize this to be an artifact of the dataset used for fine-tuning. Optimizing this process further by using fewer than ∼100B tokens, lighter fine-tuning such as LoRA [15] or tuning layer normalization [16] parameters alone would be an interesting topic for future work.

Pruning

Our pruning recipe is based on the best practices outlined in the Minitron paper [2], as described in the previous section. Specifically, for width pruning, we (1) use `l2-norm` and `mean` as the aggregation functions across the batch and sequence dimensions, respectively, and (2) perform single-shot pruning, avoiding iterative approaches. For depth pruning, we follow the observations from Gromov et al. [11] and drop a continuous subgroup of layers that results in the least accuracy drop on Winogrande [10]. In this work, we skip the lightweight neural architecture search (NAS) phase, and go with a manual architecture configuration for both LLAMA 3.1-MINITRON-4B and MN-MINITRON-8B. The architectures we come up with are inspired by the Minitron-4B and Minitron-8B models [2], and are detailed in Table 3. We provide the pruning recipes for each of our target compressed models below:

	Llama-3.1-Minitron	MN-Minitron
Peak learning rate	1e-4	1e-4
Min learning rate	1e-5	4.5e-7
Warm-up steps	40 steps	60 steps
LR decay schedule	Cosine	Cosine
Global batch size	1152	768
Context length	8192	8192
Total tokens	94B	380B

Table 4 | Hyperparameters used during distillation-based retraining.

Llama-3.1-Minitron-4B-Width:

- Starting model: Llama 3.1 8B
- Hidden dimension: 4096 → 3072
- MLP hidden dimension: 14336 → 9216
- Attention heads: unchanged
- Depth: unchanged

Llama-3.1-Minitron-4B-Depth:

- Starting model: Llama 3.1 8B
- Hidden dimension: unchanged
- MLP hidden dimension: unchanged
- Attention heads: unchanged
- Depth: 32 → 16

MN-Minitron-8B:

- Starting model: Mistral NeMo 12B
- Hidden dimension: 5120 → 4096
- MLP hidden dimension: 14336 → 11520
- Attention heads: unchanged
- Depth: unchanged

Distillation

We opt for logit-only distillation, minimizing the forward KL Divergence [12] loss across the teacher and student probabilities, and ignore the LM cross-entropy loss altogether. Here, the unpruned and pruned models correspond to the teacher and student, respectively. We use the hyperparameters listed in Table 4 during distillation. We use 32 NVIDIA DGX H100 nodes for our training jobs.

Instruction Tuning

To evaluate the instruction-following capabilities of our distilled models, we perform alignment using NeMo-Aligner [17]. We follow the same recipe for all our models by first applying math and code supervised fine-tuning (SFT) followed by instruction SFT and then two rounds of Reward-aware Preference Optimization (RPO) [18].

Analysis

We perform a series of ablation studies to better understand the effects of distillation, teacher correction, and our new depth-pruning saliency metric. We report our findings in this section.

Teacher Correction We first compare the effects of teacher correction on the MN-Minitron-8B model in Figure 4; here, we notice the clear benefits of performing teacher correction w.r.t. distilling directly from an uncorrected teacher. Next, we compare two approaches for teacher correction: (1) pruning and distilling the corrected teacher, and (2) pruning the original (uncorrected) teacher and distilling from a continuously corrected teacher. The results in Figure 5 suggest that teacher correction can be performed in parallel with distillation to recover accuracy of the pruned student model.

Pruning and Distillation Figure 6 demonstrates the orthogonal benefits of pruning and distillation over random initialization and conventional fine-tuning, respectively. We compare (1) random weight initialization and distillation, (2) random pruning and distillation, where weights are pruned randomly ignoring the importance scores, (3) our proposed pruning with typical cross entropy based LM loss training and (4) our proposed pruning with distillation-based retraining. We notice that pruning results in a significantly better starting point compared to random initialization, and distillation-based training outperforms conventional training methods. Overall, our approach requires significantly fewer training tokens (up to 40×; 380B instead of 15T tokens) to produce the state-of-the-art MN-Minitron-8B model.

Width vs. Depth Pruning Figure 7 shows the training curve of Llama 3.1-Minitron-4B pruned for width vs. depth. We notice that width pruning results in a lower initial loss and consistently outperforms the depth-pruned model, despite both variants having the same number of parameters.

Depth Pruning Metrics By examining how LM validation loss increases as contiguous blocks of layers are removed (Figure 8), we observe that the layers at the beginning and end are the most important. The figure indicates that removing non-contiguous layers can result in even better LM validation loss (the dashed line). However, we notice this observation does not necessarily hold when evaluating downstream task performance: specifically, Figure 9 shows that

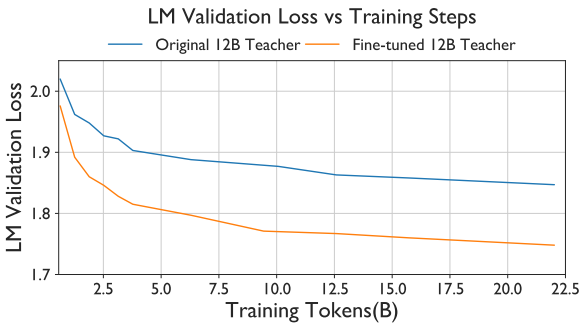

Figure 4 | Training convergence plot for the MN-MINITRON-8B student model. We compare supervision from the original teacher and the corrected teacher.

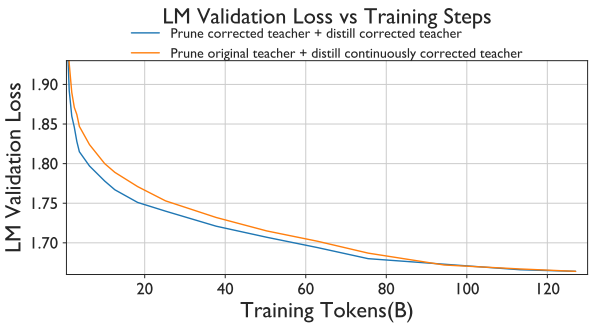

Figure 5 | Training convergence plot for the MN-MINITRON-8B student model. We compare (1) pruning and distilling the corrected teacher with (2) pruning the original (uncorrected) teacher and distilling from a continuously corrected teacher. We notice that teacher correction can be performed in parallel with distillation.

dropping 16 layers selected based on per-layer importance [8, 19] yields a random Winogrande accuracy of 0.5, while removing layers 16 to 31 continuously [11] results in an accuracy of 0.595. The gap holds during distillation-based retraining and we opt for the latter approach in this work.

Evaluation

Benchmarks following Llama [20], we evaluate our compressed base and aligned models on a series of downstream tasks, namely MMLU [21], HumanEval [22] for Python code generation, MBPP [23] and GSM8K [24]. We also evaluate the base models on several question-answering datasets for commonsense reasoning: Arc-C [25], HellaSwag [26], TruthfulQA [27], WinoGrande [10], and XL-Sum English [28] for summarization. The instruction tuned models are further evaluated for question-answering, function calling, instruction following and multiturn conversations on GPQA [29], BFCL [30], IFEval [31] and MT-Bench (GPT4-Turbo) [32], respectively. Note that this MT-Bench is a corrected version of the original MT-Bench [33].

For base models, accuracy is reported with the following evaluations settings: 5-shot on MMLU, 5-shot on Winogrande, 25-shot on ARC-Challenge, 10-shot on HellaSwag, 0-shot on 20% of XL-Sum and average pass@1 scores for HumanEval and MBPP. For pass@1 scores we use a temperature of 0.2 and nucleus sampling [34] with top-p = 0.95. For aligned models we use 0 shot and greedy sampling if applicable.

Base Models

Base model evaluation results are shown in Table 1. Compared to similarly-sized models, MN-MINITRON-8B demonstrates superior accuracy across the board, outperforming the recent Llama 3.1 8B model using 40× fewer training tokens (380B vs. 15T). Similarly, the LLAMA 3.1-MINITRON-4B models perform favorably compared to the teacher Llama 3.1 8B model using 150× fewer training tokens (94B vs. 15T); our pruned Llama models also outperform the original Minitron 4B model [2]. We note from Table 1 that the width-pruned Llama variant outperforms the depth-pruned one. These results clearly demonstrate the advantages of our methodology: state-of-the-art accuracy coupled with an order of magnitude improvement in training efficiency.

Instruct Models

The accuracy of the instruction-tuned model variants are shown in Table 2. Our aligned models outperform similarly sized variants on most evaluated benchmarks with the exception of HumanEval [35] and MBPP [23]. Additionally, LLAMA 3.1-MINITRON-4B lags behind Gemma2 on MT-Bench [33]. Nevertheless, our aligned models are consistently better on MMLU [21], GSM8K [24], GPQA [29], IFEval [31] and BFCLv2 [30]. This demonstrates the strong capabilities of our model.

Runtime Performance Analysis

To evaluate runtime performance, we optimize the Llama 3.1 8B and LLAMA 3.1-MINITRON-4B variants with NVIDIA TensorRT-LLM, an open-

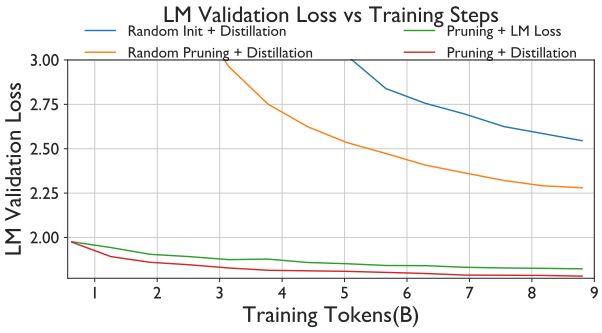

Figure 6 | Training convergence plot for the MN-MINITRON-8B model. We compare (a) random initialization with distillation, (b) randomly pruned weights with distillation, (c) pruning with standard LM loss, and (d) our pipeline with pruning and distillation. This plot shows the benefits of pruning and distillation over random initialization and conventional finetuning, respectively.

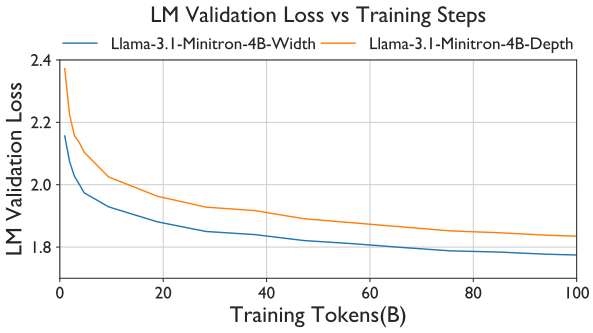

Figure 7 | Convergence plots for the width-pruned and depth-pruned versions of Llama 3.1 8B to 4B compressed models. Width pruning consistently outperforms depth pruning for a given parameter budget.

source toolkit for optimized LLM inference.

Figure 10 shows the throughput in requests per second for the various models in FP8 precision obtained on a single H100 80 GB GPU. Different use cases are represented by increasing input sequence length/output sequence length (ISL/OSL) combinations, at a batch size of 32 and 64 for the 8B-12B models and the 4B models respectively. The smaller memory footprint of the 4B model allows for larger batches. We notice that LLAMA 3.1-MINITRON-4B (Depth) is fastest, achieving an average throughput improvement of 2.7× over Llama 3.1 8B; the width-pruned variant achieves an average throughput improvement of 1.8× over Llama 3.1 8B. Compared to BF16, we notice that FP8 delivers a performance boost of 1.4×.

Insights

In this section, we summarize some interesting and surprising observations based on our evaluation.

General

1. Teacher correction is crucial for distillation to work optimally on a new, unseen dataset. Fine-tuning the teacher with the dataset used for distillation in this manner yields over a 6% reduction in LM validation loss. Teacher correction doesn't affect the optimality of pruning and can even be performed in parallel with distillation.
2. In line with the Minitron paper's observations, we require a order of magnitude fewer tokens (380B vs 15T) to achieve state-of-the-art accuracy post pruning with distillation.
3. For width pruning, we achieve stronger accuracy by retaining attention heads and pruning the other dimensions (MLP intermediate dimension, embedding channels).

Mistral NeMo 12B to MN-Minitron-8B

1. Our compressed model outperforms the teacher on two benchmarks, GSM8k and HumanEval after pruning and distillation: GSM8k increases from 55.7% to 58.5% and HumanEval increases from 23.8% to 36.2%. This improvement is likely influenced by the dataset. However, retraining is performed using the distillation loss alone.

Llama 3.1 8B to Llama 3.1-Minitron-4B

1. Width pruning delivers better accuracy with MMLU at 60.5%, while depth pruning yields 58.7%, for Llama 3.1 compression.
2. Reasoning ability for base variants appears to be impacted significantly for the depth pruned version, with GSM8K accuracy at 16.8% compared to 41.24% for the width pruned version. However, the gap reduces with instruct tuning.
3. Depth pruning boosts throughput, achieving 2.7× speedup over Llama-3.1 8B, while width pruning provides 1.7× speedup.
4. For depth pruning, we observe that dropping contiguous layers from the model is more effective than using non-contiguous, importance-based pruning.

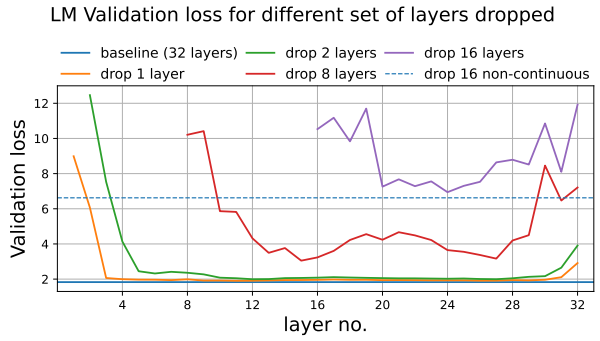

Figure 8 | LM loss value on validation set after removing 1, 2, 8 or 16 contiguous layers from Llama 3.1 8B. The purple line at layer no. 16 indicates the LM loss if we dropped the first 16 layers. Layer no. 17 indicates the LM loss if we leave the first layer intact and drop layers 2 to 17. The dashed line corresponds to LM loss value when removing 16 non-contiguous layers least increasing the loss.

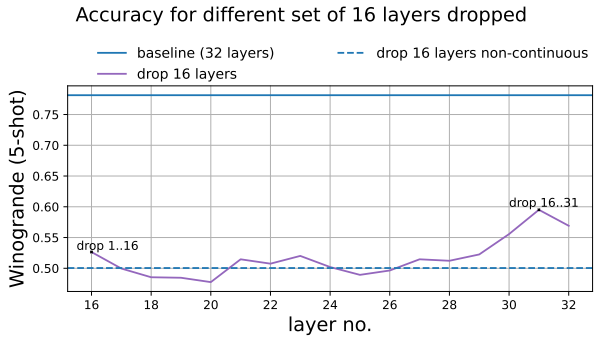

Figure 9 | Accuracy on the Winogrande task when removing 16 contiguous layers from Llama 3.1 8B. Layer no. 17 indicates the accuracy if we leave the first layer intact and drop layers 2 to 17. The dashed line corresponds to the accuracy when removing 16 non-contiguous layers that increasing the loss by the least amount.

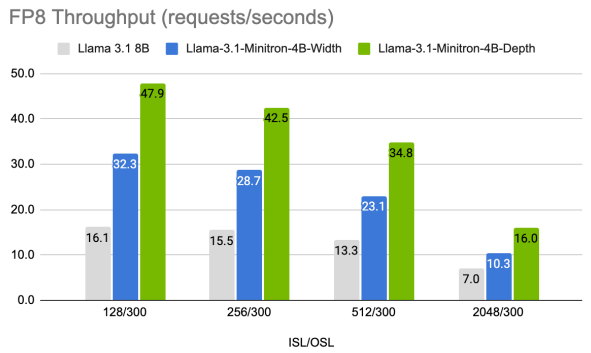

Figure 10 | TensorRT-LLM FP8 throughput comparison for the Llama 3.1-Minitron-4B models with the Llama 3.1 8B model w.r.t. increasing input and output sequence lengths.

Acknowledgments

This work would not have been possible without contributions from many people at NVIDIA. To mention a few:

Foundational Model: Sharath Turuvekere Sreenivas, Saurav Muralidharan, Raviraj Joshi, Marcin Chochowski, Pavlo Molchanov, Mostofa Patwary, Daniel Korzekwa, Ashwath Aithal, Mohammad Shoeybi, Bryan Catanzaro and Jan Kautz

Alignment: Ameya Sunil Mahabaleshwarkar, Hayley Ross, Brandon Rowlett, Oluwatobi Olabiyi, Shizhe Diao and Yoshi Suhara

Datasets: Sanjeev Satheesh, Jupinder Parmar, Shengyang Sun, Jiaqi Zeng, Zhilin Wang, Yi Dong, Zihan Liu, Rajarshi Roy, Wei Ping, Makesh Narsimhan Sreedhar and Oleksii Kuchaiev

TensorRT-LLM: Bobby Chen, James Shen and Chenhan Yu

Hugging Face Support: Ao Tang, Yoshi Suhara and Greg Heinrich

References

[1] Abhimanyu Dubey and Abhinav Jauhri et al. The Llama 3 Herd of Models. *arXiv 2407.21783*, 2024.

[2] Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. Compact language models via pruning and knowledge distillation. *arXiv preprint arXiv:2407.14679*, 2024.

[3] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*, 2015.

[4] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, Pier Giuseppe Sessa, Aakanksha Chowdhery, Adam Roberts, Aditya Barua, Alex Botev, Alex Castro-Ros, Ambrose Slone, Amélie Héliou, Andrea Tacchetti, Anna Bulanova, Antonia Paterson, Beth Tsai, Bobak Shahriari, Charline Le Lan, Christopher A. Choquette-Choo, Clément Crepy, Daniel Cer, Daphne Ippolito, David Reid, Elena Buchatskaya, Eric Ni, Eric Noland, Geng Yan, George Tucker, George-Christian Muraru, Grigory Rozhdestvenskiy,

Henryk Michalewski, Ian Tenney, Ivan Grishchenko, Jacob Austin, James Keeling, Jane Labanowski, Jean-Baptiste Lespiau, Jeff Stanway, Jenny Brennan, Jeremy Chen, Johan Ferret, Justin Chiu, Justin Mao-Jones, Katherine Lee, Kathy Yu, Katie Millican, Lars Lowe Sjoesund, Lisa Lee, Lucas Dixon, Machel Reid, Maciej Mikuła, Mateo Wirth, Michael Sharman, Nikolai Chinaev, Nithum Thain, Olivier Bachem, Oscar Chang, Oscar Wahltinez, Paige Bailey, Paul Michel, Petko Yotov, Rahma Chaabouni, Ramona Comanescu, Reena Jana, Rohan Anil, Ross McIlroy, Ruibo Liu, Ryan Mullins, Samuel L Smith, Sebastian Borgeaud, Sertan Girgin, Sholto Douglas, Shree Pandya, Siamak Shakeri, Soham De, Ted Klimenko, Tom Hennigan, Vlad Feinberg, Wojciech Stokowiec, Yu hui Chen, Zafarali Ahmed, Zhitao Gong, Tris Warkentin, Ludovic Peran, Minh Giang, Clément Farabet, Oriol Vinyals, Jeff Dean, Koray Kavukcuoglu, Demis Hassabis, Zoubin Ghahramani, Douglas Eck, Joelle Barral, Fernando Pereira, Eli Collins, Armand Joulin, Noah Fiedel, Evan Senter, Alek Andreev, and Kathleen Kenealy. Gemma: Open models based on gemini research and technology, 2024.

[5] Mistral AI team. Mistral nemo. https://mistral.ai/news/mistral-nemo, 2024. Accessed: 2024.

[6] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. Sheared llama: Accelerating language model pre-training via structured pruning. In *The Twelfth International Conference on Learning Representations*, 2023.

[7] Saleh Ashkboos, Maximilian L Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. Slicegpt: Compress large language models by deleting rows and columns. In *The Twelfth International Conference on Learning Representations*, 2023.

[8] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. ShortGPT: Layers in Large Language Models are More Redundant Than You Expect, 2024.

[9] Bo-Kyeong Kim, Geonmin Kim, Tae-Ho Kim, Thibault Castells, Shinkook Choi, Junho Shin, and Hyoung-Kyu Song. Shortened LLaMA: A simple depth pruning for large language models. In *ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2024.

[10] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. *Commun. ACM*, 64(9), 2021.

[11] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. The unreasonable ineffectiveness of the deeper layers. 2024.

[12] Solomon Kullback and Richard A. Leibler. On information and sufficiency. *Annals of Mathematical Statistics*, 22(1):79–86, 1951.

[13] Jupinder Parmar, Shrimai Prabhumoye, Joseph Jennings, Mostofa Patwary, Sandeep Subramanian, Dan Su, Chen Zhu, Deepak Narayanan, Aastha Jhunjhunwala, Ayush Dattagupta, Vibhu Jawa, Jiwei Liu, Ameya Mahabaleshwarkar, Osvald Nitski, Annika Brundyn, James Maki, Miguel Martinez, Jiaxuan You, John Kamalu, Patrick LeGresley, Denys Fridman, Jared Casper, Ashwath Aithal, Oleksii Kuchaiev, Mohammad Shoeybi, Jonathan Cohen, and Bryan Catanzaro. Nemotron-4 15b technical report, 2024.

[14] Jupinder Parmar, Sanjev Satheesh, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Reuse, don't retrain: A recipe for continued pretraining of language models, 2024.

[15] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2021.

[16] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.

[17] Gerald Shen, Zhilin Wang, Olivier Delalleau, Jiaqi Zeng, Yi Dong, Daniel Egert, Shengyang Sun, Jimmy Zhang, Sahil Jain, Ali Taghibakhshi, Markel Sanz Ausin, Ashwath Aithal, and Oleksii Kuchaiev. Nemo-aligner: Scalable toolkit for efficient model alignment, 2024.

[18] Nvidia, :, Bo Adler, Niket Agarwal, Ashwath Aithal, Dong H. Anh, Pallab Bhattacharya, Annika Brundyn, Jared Casper, Bryan Catanzaro, Sharon Clay, Jonathan Cohen, Sirshak Das, Ayush Dattagupta, Olivier Delalleau, Leon Derczynski, Yi Dong, Daniel Egert, Ellie Evans, Aleksander Ficek, Denys Fridman, Shaona Ghosh, Boris Ginsburg, Igor Gitman, Tomasz Grzegorzek, Robert Hero, Jining Huang, Vibhu Jawa, Joseph Jennings, Aastha Jhunjhunwala, John Kamalu, Sadaf Khan, Oleksii Kuchaiev, Patrick LeGresley, Hui Li, Jiwei Liu, Zihan Liu, Eileen Long, Ameya Sunil Mahabaleshwarkar, Somshubra Majumdar, James Maki, Miguel Martinez, Maer Rodrigues de Melo, Ivan Moshkov, Deepak Narayanan, Sean Narenthiran, Jesus Navarro, Phong Nguyen, Osvald Nitski, Vahid Noroozi, Guruprasad Nutheti, Christopher Parisien, Jupinder Parmar, Mostofa Patwary, Krzysztof Pawelec, Wei Ping, Shrimai Prabhumoye, Rajarshi Roy, Trisha Saar, Vasanth Rao Naik Sabavat, Sanjeev Satheesh, Jane Polak Scowcroft, Jason Sewall, Pavel Shamis, Gerald Shen, Mohammad Shoeybi, Dave Sizer, Misha Smelyanskiy, Felipe Soares, Makesh Narsimhan Sreedhar, Dan Su, Sandeep Subramanian, Shengyang Sun, Shubham Toshniwal, Hao Wang, Zhilin Wang, Jiaxuan You, Jiaqi Zeng, Jimmy Zhang, Jing Zhang, Vivienne Zhang,

Yian Zhang, and Chen Zhu. Nemotron-4 340b technical report, 2024.

[19] Shoaib Ahmed Siddiqui, Xin Dong, Greg Heinrich, Thomas Breuel, Jan Kautz, David Krueger, and Pavlo Molchanov. A deeper look at depth pruning of llms. *arXiv preprint arXiv:2407.16286*, 2024.

[20] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *ArXiv*, abs/2307.09288, 2023.

[21] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

[22] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, Suchir Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374, 2021.

[23] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.

[24] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[25] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *ArXiv*, abs/1803.05457, 2018.

[26] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy, July 2019. Association for Computational Linguistics.

[27] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods, 2022.

[28] Tahmid Hasan, Abhik Bhattacharjee, Md Saiful Islam, Kazi Samin, Yuan-Fang Li, Yong-Bin Kang, M. Sohel Rahman, and Rifat Shahriyar. Xl-sum: Large-scale multilingual abstractive summarization for 44 languages, 2021.

[29] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023.

[30] Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Berkeley function calling leaderboard. 2024.

[31] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

[32] Zhilin Wang, Yi Dong, Olivier Delalleau, Jiaqi Zeng, Gerald Shen, Daniel Egert, Jimmy J. Zhang, Makesh Narsimhan Sreedhar, and Oleksii Kuchaiev. Helpsteer2: Open-source dataset for training top-performing reward models, 2024.

[33] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc., 2023.

[34] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *ArXiv*, abs/1904.09751, 2019.

[35] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.